Ders 3 Genişletilmiş Notu: Bağlantı Problemi, Öklid Algoritması ve Algoritma Analizinin Temelleri

Ders 3 Genişletilmiş Notu: Bağlantı Problemi, Öklid Algoritması ve Algoritma Analizinin Temelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Haftanın Tekrarı
- 2. Bağlantı Problemi
- 3. Neden Verimli Algoritma Tasarlarız?
- 4. Algoritma Tasarım Teknikleri
- 5. Ortak Bölenlerin En Büyüğü ve Öklid Algoritması
- 6. Eratosthenes Kalburu'nun Tekrarı
- 7. Algoritma Analizine Giriş
- 8. İşlem Maliyetlerinin Sayılması
- 9. Büyüme Oranı
- 10. Yaygın Büyüme Oranları ve Döngü Örnekleri
- 11. Big O Gösterimi
- 12. Big Omega ve Big Theta
- 13. Big O'nun Özellikleri ve Örnekler
- 14. En Kötü, En İyi ve Ortalama Durum
- 15. Sıralı Arama ve İkili Arama
- 16. Sıralama Algoritmalarının Karmaşıklığı
- 17. Logaritmik Karmaşıklığın Gücü

1. Geçen Haftanın Tekrarı

Hoca derse geçen haftayı kısaca hatırlatarak başladı. İlk derste dersin planı, kaynakları ve değerlendirme ölçütleri konuşulmuştu. Sonra şu sorulara bakılmıştı: Algoritma nedir? Bir algoritmanın özellikleri nelerdir? Hangi şeyler algoritmadır, hangileri değildir? Ardından veri nedir, veri nasıl tutulur, veri yapısı nedir ve neden gereklidir sorularıyla derse giriş yapılmıştı.

Algoritma tasarımının, işlemle çözülebilen problemlere karşı geliştirdiğimiz çözüm yolları olduğunu söylemiştik. Bu hafta hoca, "biraz beyin jimnastiği yaparak" doğrudan bir örnekle başladı. Amaç şunu göstermekti: Neden algoritma tasarlarız, veri yapısı kullanmak işi nasıl kolaylaştırır ve tasarımımızı nasıl değiştirir?

2. Bağlantı Problemi

2.1 Problemin Tanımı

Buradaki "bağlantı", "hocam bağlanamıyorum, hat çekmiyor" türünden bir sorun değildir. Bağlantı problemi (connectivity problem) şöyle tanımlanır: Girdi olarak, birbiriyle bağlantılı varlık çiftleri sırayla verilir. Örneğin `p-q` çifti, "p ile q birbirine bağlıdır" demektir. Bağlantının yönlü olmadığını varsayıyoruz. Bu durumda q da doğal olarak p'ye bağlıdır.

Bu varlıklar birer isim olabilir: sosyal medyadaki kullanıcı adları, bir bilgisayar ağındaki düğümlerin kimlikleri (ID) ya da adları gibi. Kolaylık için hepsini tam sayıyla gösteriyoruz. Amacımız yalnız **yeni olan**, önceden var olmayan bağlantıları bulmaktır.

Hoca örneği tahtaya çizer gibi adım adım kurdu:

- `3-4` geldi: 3 numaralı varlık ile 4 numaralı varlık bağlıdır.
- `4-9` geldi: 4 ile 9 da bağlıdır. Artık 3 ile 9 da dolaylı olarak bağlıdır.
- `8-0` geldi: 8 ile 0 bağlandı.
- `2-3` geldi: 2 de 3'e bağlandı.
- `5-6` geldi: 5 ile 6 bağlandı.
- `2-9` geldi: Burada durmamız gerekir. 2-3, 3-4 ve 4-9 bağlantıları zaten var. Yani 2 ile 9 zaten bağlıdır. Yeni bir bağ kurulmamalı, algoritma "2, 3, 4, 9 zaten bağlı" diye bunu tespit edebilmelidir. Slaytta bu çift kırmızıyla yazılmıştı.

2.2 Uygulama Alanları

Hoca problemi somutlaştırmak için birkaç gerçek hayat örneği verdi:

- **Bilgisayar ağı:** Düğümler ya da bilgisayarlar birbirine paket yollamak istiyor. p ile q arasında her seferinde yeni bir bağlantı mı kurmalıyız, yoksa zaten bağlılar mı ve paketler mevcut yol üzerinden iletilebilir mi? Ağda milyonlarca, milyarlarca cihaz olabilir.
- **Baskılı devre:** Elinizde tellerle birbirine bağlı kontakları olan bir elektronik devre var. Gereksiz bağlantı yapmadan bütün uçları nasıl bağlarız? Bu problem de aynı yöntemle ele alınabilir.
- **Sosyal ağ:** İki kişi arasında, "arkadaşının arkadaşının arkadaşı" üzerinden bir bağlantı kurulabiliyor mu?
- **Programdaki değişken adları** gibi başka pek çok durum.

Slaytta, kitaptan alınmış büyük bir ağ görseli vardı. Hoca görseldeki iki düğümün bağlı olup olmadığını sordu. Öğrenciler "bağlıdır" dedi. Hoca bir yolu çizerek bağlı olduklarını gösterdi ve şunu vurguladı: Burada yalnız **bağlı mı, değil mi** sorusuyla ilgileniyoruz, **nasıl bağlı** oldukları ile ilgilenmiyoruz.

2.3 Problemi Makul Biçimde Tanımlamak

Hoca algoritma tasarımının ilk adımının, çözülecek problemi makul, anlaşılır ve açıklanabilir biçimde tanımlamak olduğunu söyledi. Problemi tam haliyle çözmek maliyetli ya da zor olabilir. Bu durumda problem tanımını, orijinalinden biraz farklı da olsa işe yarar bilgi veren bir hâle getirebiliriz.

Örneğin burada "p ile q **hangi yolla** bağlıdır?" sorusunu cevaplamıyoruz. Bu da mümkündür, ama daha karmaşık bir algoritma ve graf veri yapısı gerekir. Hiç graf bilmeden, yalnız "bağlı mı?" sorusunu cevaplayan daha basit bir algoritmayı hızlıca tasarlayabiliriz. Bu cevap orijinal probleme de destek olur.

> **Hocanın vurgusu:** Problem çok zorsa, işe yarar bilgiyi veren daha basit bir probleme dönüştürülebilir. Hoca bunun özellikle büyük problemlerde sıkça yapıldığını söyledi.

Hoca ayrıca n nesneyi bağlamak için kaç bağlantı gerektiğini sordu. n nesneyi bağlamak için $n - 1$ bağlantı yeterlidir. Bu konu ileride minimum yayılan ağaç (minimum spanning tree) konusunda doğrudan görülecek.

Özetle şu anda tek bir soruya true ya da false cevabı vermeye çalışıyoruz: Verilen çift bağlı mıdır, değil midir?

2.4 Hangi Veri Yapısı?

Hoca "nasıl çözebiliriz?" diye sordu. Bir öğrenci önce çiftlerin nasıl geldiğini sordu.

Soru: Bu bağlantıları baştan biz mi kuruyoruz, yoksa herhangi bir bağlantıyı mı sorguluyoruz?
Cevap: Çiftler bir yerden sırayla okunur. Örneğin kullanıcı sırayla 2-9, 4-8, 1-6, 0-2 girer, ya da bir listeden veya diziden okunur. Fark etmez, sırayla gelirler.

Ardından bir öğrenci bir çözüm önerdi: Bir matris tutulur, gelen bağlantılar koordinat olarak matriste 1 ile işaretlenir, sonra matris üzerinde bir yol arama algoritmasıyla iki düğüm arasında yol olup olmadığı sorgulanır. Hoca bunun bir yol olduğunu ve büyük problemlerde var olan bir algoritmayı ara çözüm olarak kullanmanın sık yapıldığını kabul etti. Ama bu yöntemle her sorguda ayrı bir algoritma koşturmak gerekir. Hızlı değildir. Başka bir öğrenci pointer'larla bağlı bir yapı kurmayı önerdi.

Hoca doğru düşünce yolunu şöyle kurdu: Her yeni çift geldiğinde, bu ikisinin zaten bağlı olup olmadığını kontrol etmeliyiz. Bunun için önceki bağlantıları bir şekilde **hatırlamamız**, bir yerde tutmamız gerekir. Nerede? Bir veri yapısında. O hâlde ilk düşünmemiz gereken şey şudur: Bu algoritmayı etkin biçimde tasarlamak için hangi veri yapısı uygundur?

Bu veri yapısı, birbiriyle bağlı **kümeleri** tutmalıdır. Algoritmanın fikri şöyledir:

- Gelen çiftin birinci elemanının ait olduğu küme bulunur.
- İkinci elemanın ait olduğu küme bulunur.
- İki küme aynıysa, bu elemanlar zaten bağlıdır. Hiçbir şey yapılmaz.
- Kümeler farklıysa, iki küme birleştirilir.

Sonunda oluşan her küme bir bağlı bileşen (connected component) olur.

2.5 Dizi ile Çözüm

Hoca önce sonucu gösterdi. Slayttaki son durumda 0, 5 ve 6 numaralı düğümler aynı küme numarasına (0) sahipti. 1, 2 ve 7 numaralı düğümler 1 numaralı kümedeydi. 8, 3, 4 ve 9 ise 8 numaralı kümedeydi. "7, 1'e bağlı mı?" sorusunu cevaplamak için 1'in küme değerine (1) ve 7'nin küme değerine (1) bakılır. Aynı oldukları için bağlıdırlar. "1, 8'e bağlı mı?" sorusunda 1'in kümesi 1, 8'in kümesi 8'dir. Farklı oldukları için bağlı değildirler.

Bu sonuca nasıl ulaşılır? Hoca algoritmayı akış diyagramıyla değil, adım adım anlattı. 10 düğüm vardır ve 10 elemanlı **tek bir dizi** (`id`) tutulur. Başlangıçta her düğüm kendi kümesindedir: `id[i] = i`. Yani başta birbirinden ayrık 10 küme vardır.

`p-q` geldiğinde kural şudur: `id[p] == id[q]` ise hiçbir şey yapılmaz. Değilse dizi taranır ve değeri `id[p]` olan **bütün** elemanlar `id[q]` yapılır. Yani "p'yi, q'nun ait olduğu kümeye koy".

Hocanın izlediği adımlar:

- **4-3:** 4'ün değeri 3 yapılır. Artık 3 ile 4'ün küme kimlikleri aynı.
- **3-8:** 8'in değeri 8'dir. Dizi taranır, bütün 3'ler 8 yapılır. Böylece 3, 4 ve 8 aynı kümede olur.
- **6-5:** Bütün 6'lar 5 yapılır.
- **9-4:** 4'ün değeri 8'dir. 9'un değeri 9'dur. Bütün 9'lar 8 yapılır. Şu anda bir ikili, bir dörtlü ve dört tane tek elemanlı küme vardır; toplam 6 küme.
- **2-1:** Bütün 2'ler 1 yapılır.
- **8-9:** Normalde dizi taranacaktı. Ama ikisinin değeri de 8'dir. "Bunlar zaten aynı kümede" denir ve hiçbir şey yapılmaz.
- **5-0:** Bütün 5'ler 0 yapılır. 5 ile 6 zaten bağlıydı, ikisi de 0 olur. Hoca burada slayttaki bir hatayı fark edip düzeltti ve öğrencilere "Uyuyor musunuz arkadaşlar? Söylesenize hatayı" diye takıldı.
- **7-2:** Yalnız kalan 7 de bağlanır. 2'nin değeri 1'dir, bütün 7'ler 1 yapılır.
- **6-1:** 6'nın değeri 0'dır. Bütün 0'lar 1 yapılır.

Dizinin her adımdaki hâli (C ile doğrulandı):

Gelen çift	0	1	2	3	4	5	6	7	8	9
Başlangıç	0	1	2	3	4	5	6	7	8	9
4-3	0	1	2	3	3	5	6	7	8	9
3-8	0	1	2	8	8	5	6	7	8	9
6-5	0	1	2	8	8	5	5	7	8	9
9-4	0	1	2	8	8	5	5	7	8	8
2-1	0	1	1	8	8	5	5	7	8	8
8-9	değişiklik yok									

Gelen çift	0	1	2	3	4	5	6	7	8	9
5-0	0	1	1	8	8	0	0	7	8	8
7-2	0	1	1	8	8	0	0	1	8	8
6-1	1	1	1	8	8	1	1	1	8	8

Son durumda iki bileşen kalır: 6 düğüm (0, 1, 2, 5, 6, 7) 1 numaralı kümededir, 4 düğüm (3, 4, 8, 9) 8 numaralı kümededir.

Hocanın adım adım anlattığı algoritma C ile şöyle yazılabilir:

```c
#include <stdio.h>
#define N 10

int main(void) {
    int id[N], i, k, p, q, t;
    int pairs[][2] = {{4,3},{3,8},{6,5},{9,4},{2,1},{8,9},{5,0},{7,2},{6,1}};

    for (i = 0; i < N; i++)          /* her düğüm kendi kümesinde */
        id[i] = i;

    for (k = 0; k < 9; k++) {
        p = pairs[k][0];
        q = pairs[k][1];
        if (id[p] == id[q]) {        /* aynı kümede: zaten bağlı */
            printf("%d-%d zaten bagli\n", p, q);
            continue;
        }
        t = id[p];
        for (i = 0; i < N; i++)      /* p'nin kümesindekileri q'nun kümesine taşı */
            if (id[i] == t)
                id[i] = id[q];
    }
    return 0;
}
```

2.6 Değerlendirme

Hoca çözümü patron örneğiyle bağladı: Patron “bunlar bağlı mı değil mi, bana hızlı ve etkin biçimde söyleyeceksin” diyor. Çiftler sırayla geliyor ve her anda bağlı olup olmadıklarını takip edebilmemiz gerekiyor. Matris ve yol arama önerisi bir yöntemdir, ama her seferinde başka bir algoritma koşturulur. Onun yerine çok basit bir veri yapısı, yalnızca bir dizi kullanıldı ve kümelerin kimlikleri tutuldu. Bütün olay bu kadar. Değerlerin sayı olması da zorunlu değildir; kolay anlatmak için sayı kullanıldı.

Hoca bunu “algoritma tasarımı nedir, problem nasıl ele alınır, hangi veri yapısı nasıl kullanılır” sorularının çok basit ama çok iyi çalışan bir örneği olarak sundu.

> **Hocanın vurgusu:** Böyle çözümleri ilk denemede bulamazsınız. Bulabiliyorsanız ya çok şanslısınızdır ya da çok iyisinizdir; genelde ikisi de olmaz. O yüzden çok çalışmak gerekir. İnternetteki coding challenge ve algoritma problemi sitelerine girin, problemi alın, hazır çözüme bakmadan önce kendiniz bir yol geliştirin. Sonra hazır çözümlerle karşılaştırın: “Bu bunu nasıl yapmış, bu neden aklıma gelmedi, şu veri yapısını kullanmam ne kadar iyi olmuş” gibi.
>
> **Ek bilgi (derste anlatılmadı):** Derste anlatılan dizi tabanlı çözüm, literatürde union-find (ayrık kümeler, disjoint set) yapısının “quick-find” adlı sürümü olarak bilinir. Bu sürümde “bağlı mı?” sorgusu iki dizi okumasıyla $O(1)$ sürede cevaplanır. Ama her birleştirme bütün diziyi taradığı için $O(n)$ sürer. Kümeleri ağaç olarak tutan quick-union ve ağırlıklı sürümler birleştirmeyi çok daha ucuz yapar.

3. Neden Verimli Algoritma Tasarlarız?

Hoca teorik tarafa geçmeden önce basit bir örnek verdi. Elimizde 3000 mektup var. Bunları log dosyası gibi de düşünebiliriz. Her satırda 10 kelime, her mektupta 20 satır var. Her mektupta $10 \cdot 20 = 200$ kelime vardır. 3000 mektupta toplam $3000 \cdot 200 = 600\,000$ kelime (ya da kayıt) bulunur. Kelimeler tekrarlı olabilir; bazıları çok sık tekrar edebilir. Hoca verinin doğasının da algoritma tasarımını etkilediğini belirtti.

Hoca kendi adını, "Göksel" kelimesini aradığını varsaydı. Bu kelimeyle eşleşen bütün kelimeleri bulmak ya da saymak istiyoruz. Varsayımlar:

- Bilgisayarın hızıyla her kelime karşılaştırması ("buna eşit mi?") 1 saniye sürsün.
- Bütün 600 bin kelime, birisi tarafından önceden A'dan Z'ye sıralanmış olsun.

Sıralı arama: Birinci mektubun birinci satırındaki birinci kelimeden başlayıp 600 bininci kelimeye kadar sırayla karşılaştırırsak 600 000 karşılaştırma yaparız:

$$600\,000 \text{ s} = \frac{600\,000}{3600} \text{ saat} \approx 166{,}7 \text{ saat} \approx 6{,}9 \text{ gün}$$

Yani neredeyse bir hafta.

İkili arama: Girdinin sıralı olduğu bilgisini kullanırsak (exploit edersek), yani "bu bilgiyi sömürerek" ikili arama (binary search) kullanırsak yaklaşık $\log_2 600\,000$ karşılaştırma gerekir:

$$\log_2 600\,000 \approx 19{,}2 \;\Rightarrow\; \text{yaklaşık } 20 \text{ karşılaştırma} \approx 20 \text{ s}$$

Aynı sonucu bir yöntem 166 saatte, öbürü 20 saniyede üretiyor. Tabii ki ikincisini tercih ederiz. Bu yüzden problemi etkin, iyi ve hızlı çözen algoritmalar tasarlamaya çalışırız.

4. Algoritma Tasarım Teknikleri

Hoca algoritma tasarlamanın pek çok tekniği ve stratejisi olduğunu söyleyip bunları hızlıca saydı. Ayrıntıları 3. sınıftaki Algoritma Analizi dersinde görülecek.

- **Kaba kuvvet (brute force):** Bazı problemleri maalesef yalnız bu şekilde çözebiliriz. Örnek: Şifresini unuttuğunuz, RAR ile sıkıştırılmış bir dosya. Tek seçenek bütün şifre kombinasyonlarını tek tek denemektir.
- **Açgözlü (greedy):** Çözümün her adımında o adım için en iyi olan seçilir. Hocanın örneği: Bir labirentten çıkmaya çalışıyorsunuz ve önünüzde üç kapı var. Birinin önünde 1, birinin önünde 5, birinin önünde 3 elma var. Daha çok enerji alacağınız için 5 elmalı kapıya girersiniz. Kapıdan sonrasına girdikten sonra bakarsınız. Bu yaklaşımın belli kısıtları ve şartları vardır.
- **Böl ve yönet (divide and conquer):** Büyük problem parçalara ayrılır, parçalar çözülür, sonra çözümler birleştirilir. Aslında böl, yönet ve birleştir (divide, conquer, combine) olmak üzere üç aşamadır. En bilinen örnek merge sort'tur.
- **Azalt ve yönet (decrease and conquer):** Arama uzayı adım adım küçültülerek sonuca ulaşılır. Örnek: insertion sort. Bu derste karmaşıklık analizi için üzerinden geçilecek.
- **Dönüştür ve yönet (transform and conquer):** Problem, çoğunlukla bir veri yapısı kullanılarak başka bir yapıya dönüştürülür. Dönüştürülmüş problem çözülünce orijinal problem de çözülmüş olur.
- **Dinamik programlama (dynamic programming):** Adındaki "programlama"ya bakmayın, aslında bir optimizasyon tekniğidir. Fibonacci sayılarının hesabı üstel karmaşıklığa sahip olabilir. Bunu doğrusal karmaşıklıkla çözmenin yolu dinamik programlama gibi yaklaşımlardan geçer. Hoca bunu gelecek hafta bir örnekle detaylandıracağını söyledi.
- **Yinelemeli iyileştirme (iterative improvement):** Sayısal analizdeki Newton-Raphson kök bulma yöntemi buna çok iyi bir örnektir.
- **Geri izleme (backtracking):** Sudoku çözmede kullanılan teknik.

5. Ortak Bölenlerin En Büyüğü ve Öklid Algoritması

5.1 Ortaokul Yöntemi

Hoca ortaokuldan beri bildiğimiz bir problemi ele aldı: ortak bölenlerin en büyüğü (OBEB, greatest common divisor). İki doğal sayı m ve n verildiğinde, ikisini de bölen sayıların en büyüğünü bulmak istiyoruz. Ortaokulda öğretilen yöntem şudur:

- m'yi asal çarpanlarına ayır.
- n'yi asal çarpanlarına ayır.
- m ve n'nin bütün ortak asal çarpanlarını bul.
- Ortak asal çarpanları birbiriyle çarp.
- Çarpımı OBEB olarak ver.

Hoca örnek olarak 6 ve 18'i aldı.

- 6'nın asal çarpanları: 2 ve 3. Bir öğrenci 1'i de saymak istedi. Başka bir öğrenci 1'in asal olmadığını söyledi. Hoca "neden değil, kendisinden ve birden başka böleni yok, tanıma uymuyor mu?" diye sordu. Öğrencinin cevabı: Asal sayı tanımında sayının 2 ya da 2'den büyük olması gerekir.
- 18'in asal çarpanları: $18 = 2 \cdot 3^2$. Yani 2 ve 3.
- Ortak asal çarpanlar: 2 ve 3. Çarpımları $2 \cdot 3 = 6$. OBEB $= 6$.

Sonra hoca sordu: Bu bir algoritma mı?

Soru: Bu yöntem bir algoritma mıdır? **Cevap:** Bir öğrenci "girişi ve sonucu olduğu için algoritma sayılabilir, ama etkinliği tartışılır" dedi. Hoca etkinliği sormadığını, algoritma tanımına uyup uymadığını sorduğunu söyledi. Algoritmada adımların açık, kesin ve tutarlı olması gerekir. Burada ilk adımda "m'nin asal çarpanlarını bul" deniyor, ama asal çarpanların nasıl bulunacağı söylenmiyor. Adım bir fonksiyon gibi geçiyor, ayrıntısı yok. Girdisi var, çıktısı var, adımları "kör topal" var, ama içerikleri muğlak. Bu hâliyle bir algoritma değildir.

> **Hocanın vurgusu:** Bir algoritmanın her adımı açık ve tanımlı olmalıdır. "Asal çarpanlarını bul" diyen ama bunun nasıl yapılacağını söylemeyen bir yöntem algoritma sayılmaz.

5.2 Öklid Algoritması

Bu problemi Öklid, Elements kitabında MÖ 300 civarında, yani yaklaşık 2300 yıl önce tanımlamıştır. Kural şudur:

$$\gcd(m, n) = \gcd(n,\ m \bmod n)$$

Yani iki sayının OBEB'i, ikinci sayı ile birinci sayının ikinciye bölümünden kalanın OBEB'ine eşittir. Bu işlem $n = 0$ olana kadar tekrarlanır. O anda elde kalan m sayısı OBEB'dir.

Örnek 1: $\gcd(60, 24)$

- $60 \bmod 24$: 24'ün katları 24 ve 48'dir. $60 - 48 = 12$. Kalan 12. Yani $\gcd(60, 24) = \gcd(24, 12)$.
- $24 \bmod 12 = 0$. Yani $\gcd(24, 12) = \gcd(12, 0)$.
- $n = 0$ oldu. Elde kalan $m = 12$.

$$\gcd(60, 24) = \gcd(24, 12) = \gcd(12, 0) = 12$$

Hoca bu yöntemin güzelliğinin, sayılar ne olursa olsun her zaman çalışması olduğunu söyledi. Bunu ispatlamak gerekir. Hoca ispatı derste yapacak vakti olmadığını, mutlaka araştırılmasını söyledi.

Örnek 2: $\gcd(1785, 546)$

Bu sayıları 6 ve 18 kadar kolay asal çarpanlarına ayıramayız. Öklid yöntemiyle (python ile doğrulandı):

m	n	m mod n
1785	546	147
546	147	105
147	105	42
105	42	21
42	21	0
21	0	sonuç: 21

Sonuç: $\gcd(1785, 546) = 21$.

> **Ders sonrası not:** Hoca sözlü anlatımda "546 mod 147'den 105" adımından sonra doğrudan "105 mod 42'den 21" adımına geçti. Aradaki $147 \bmod 105 = 42$ adımı atlandı. Tam adımlar yukarıdaki tabloda (ara adım eklendi).

5.3 Sözde Kod ve Akış

Slaytta algoritma iki biçimde gösterildi. Solda başlangıç ve bitiş sembolleri olmayan basit bir sözde kod vardı:

```
Adım 1: n = 0 ise m'yi döndür ve dur; değilse Adım 2'ye geç.
Adım 2: m'yi n'ye böl, kalanı r'ye ata.
Adım 3: m ← n, n ← r. Adım 1'e dön.
```

Sağda ise akış diyagramı vardı. Hoca her algoritmanın bir yerde başlaması ve sonlu sayıda adımdan sonra bitmesi gerektiğini hatırlattı. Akış şöyledir: Başla. Okuma (girdi) sembolüyle m ve n okunur. $n \neq 0$ olduğu sürece bir `while` döngüsü döner. Döngü gövdesinde önce kalan bulunur (`r`, remainder'dan gelir): `r = m mod n`. Sonra `m = n`, ardından `n = r` yapılır. Döngü başına dönülür ve n yeniden kontrol edilir. $n = 0$ olduğu anda daha fazla bölmeye gerek yoktur. Döngüden çıkılır, `m` yazdırılır ve algoritma durur. Hoca akış oklarını PowerPoint'te düzgün çizemediğini, elle çizse kırık çizgili bir okla döngüyü göstereceğini söyledi.

Akışın C karşılığı (gcc ile derlendi ve iki örnekle doğrulandı):

```c
int gcd(int m, int n) {
    int r;
    while (n != 0) {
        r = m % n;
        m = n;
        n = r;
    }
    return m;
}
```

Soru: m ile n'nin büyük olanını bulmamız gerekmiyor mu? **Cevap:** Hoca gerektiğini söyledi. Hızlı anlatmak için birinci sayının ikinciden büyük olduğu varsayılmıştı. Aksi hâlde başta ekstra bir `if` ve bir yer değiştirme (swap) işlemi gerektiğini söyledi.

> **Ders sonrası not:** Hoca $m < n$ durumunda ek bir yer değiştirme gerektiğini söyledi; aslında gerekmez. $m < n$ ise ilk adımda $m \bmod n = m$ olur. Böylece `m = n, n = m` atamaları iki sayıyı kendiliğinden yer değiştirir. Örneğin yukarıdaki kodda `gcd(24, 60)` çağrısı da 12 döndürür (derleyiciyle denendi).

Hoca algoritmanın teorik olarak doğru çalıştığını ispatlamak gerektiğini, bunun için de n'nin mutlaka bir noktada 0 olacağını göstermek gerektiğini söyledi ve bunu sınıfa sordu. Soru ve cevabı "Örnek Sorular" bölümünde.

Hoca ayrıca Öklid algoritmasının "asal çarpanlara ayırmakla neredeyse birebir aynı şeyi yaptığını" söyledi.

> **Ders sonrası not:** Öklid algoritması asal çarpanlara ayırma yapmaz. Yalnız bölme ve kalan alma işlemleriyle ilerler. Sonucu ortaokul yöntemiyle aynıdır, ama yöntem farklıdır. Asal çarpanlarına kolay ayrılamayan büyük sayılarda Öklid'in tercih edilmesinin sebebi de budur.

6. Eratosthenes Kalburu'nun Tekrarı

Arada hoca geçen hafta görülen asal sayı bulma problemini tekrar etti: n'ye kadar olan bütün asal sayılar etkin biçimde nasıl bulunur? Bunun için Eratosthenes Kalburu (Sieve of Eratosthenes) örneği verilmişti. 2'nin katları, 3'ün katları, sonra sıradaki asal sayının katları silinerek gidilir. $\sqrt{n}$'ye kadar gitmek yeterlidir, çünkü daha büyük bir sayının karesi zaten n'yi aşar. $\sqrt{n}$'ye kadar olan asal sayıların bütün katları silinince asal olmayanlar silinmiş olur. Geriye kalan her şey asaldır.

Slaytta 1'den 100'e kadar sayılar matris şeklinde yazılmıştı. Hoca bunun aslında matris değil, slayta sığsın diye bölünmüş tek boyutlu bir dizi olduğunu (1, 2, 3, ..., 15, ... diye dümdüz giden) söyledi. Örneği 36'ya kadar yaptı:

- $\sqrt{36} = 6$. 6'ya kadar gitmek yeter.
- 2 asaldır. 2'nin katları silinir: 4, 6, 8, ..., 34, 36.
- 3 asaldır. 3'ün katları silinir: 9, 15, 21, 27, 33 silinir; 6, 12, 18, 24, 30, 36 zaten silinmişti.
- 4 zaten silinmişti.
- 5'in katları: 10, 15, 20, 25, 30, 35. Bunlardan 25 ve 35 bu adımda silinir, diğerleri zaten silinmişti.
- 6 zaten silinmişti; katları (12, 18, 24, 30, 36) da silinmişti.

Kalanlar (python ile doğrulandı): 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31.

Slayttaki akış şöyleydi: Önce 2'den n'ye kadar bütün değerler bir `A` dizisine yazılır. Sonra p, 2'den $\sqrt{n}$'ye kadar gider. `A[p]` daha önce sıfırlandıysa hiçbir şey yapılmaz. Sıfırlanmadıysa $j = p \cdot p$'den başlanır ve $j \leq n$ olduğu sürece j her adımda p artırılarak katlar sıfırlanır.

Hoca bunu böyle çözmek zorunda olmadığımızı, farklı bir gösterim de kullanılabileceğini söyledi. Sayılara zaten dizi indisleriyle erişebiliriz. O hâlde sayıları ayrıca saklamak yerine, dizide yalnız 1 (asal) ya da 0 (asal değil) tutulabilir. Başta hepsi asal kabul edilir (hepsi 1). Henüz silinmemiş her i için, $j = i$'den başlayıp $i \cdot j \leq n$ olduğu sürece `A[i * j] = 0` yapılır. Yani 2 için 4, 6, 8, 10 sıfırlanır, sonra 3 için 9, 12, ... sıfırlanır.

```
/* A[2..n] başta 1; döngü sonunda A[k] == 1 ise k asaldır */
for (i = 2; i <= n; i++)
    A[i] = 1;
for (i = 2; i * i <= n; i++)
    if (A[i])
        for (j = i; i * j <= n; j++)
            A[i * j] = 0;
```

Bu kod $n = 36$ için derlenip çalıştırıldı ve yukarıdaki 11 asal sayıyı verdi.

Bu gösterimin kazancı nedir? Örneğin gömülü bir sistemde bu değerler byte ya da integer olarak değil, **bit** olarak saklanabilir. Böylece çok büyük bir yer kazancı elde edilir. Yapılan iş aynıdır; değişen yalnız verinin gösterimidir. İndisler sayıları zaten gösterir, dizi yalnız "asal mı, değil mi" bilgisini tutar.

7. Algoritma Analizine Giriş

Aradan sonra hoca algoritma analizinin temellerine geçti. Algoritma analizi uzun, derin ve zor bir konudur. Bu derste, algoritmaları karşılaştırmaya yetecek kadar yüzeysel biçimde görülecek. Ayrıntısı Algoritma Analizi dersinde gelecek.

Algoritma analizi bilgisayar bilimlerinde bir çalışma alanıdır. Amacı, farklı çözüm yöntemlerinin başarımını (performance) analiz etmek ve karşılaştırmak için araçlar sunmaktır. Algoritmaları üç sebeple analiz ederiz:

- **Başarımı ölçmek:** Bir algoritma ne kadar iyi, ne kadar hızlı sonuç üretiyor?
- **Karşılaştırmak:** Aynı problemi çözen farklı yöntemleri karşılaştırmak. Örneğin kelime arama örneğinde, aynı ortamda biri 166 saat, öbürü 20 saniye sürüyordu.
- **Daha iyisini aramak:** Elimizdeki çözümün daha iyisi mümkün mü? Bu, olabilecek çözümlerin en iyisi mi?

Aynı problemi çözen iki algoritmadan hangisinin daha iyi olduğunu anlamak için iki temel yöntem vardır: deneysel (empirical) analiz ve teorik analiz.

7.1 Deneysel Analiz

Empirik, gözleme dayalı demektir. Daha basit ve ilkel bir yöntemdir. İki algoritma bir programlama diliyle gerçekleştirilir (implementation), çalıştırılır ve işletim süreleri karşılaştırılır. Birinci algoritma başlarken kronometreye basılır, algoritma sonucu üretip durunca bir daha basılır. İkincisi için de aynısı yapılır. Biri 20 saniyede, öbürü 140 saniyede sonuç üretmişse birincisi daha başarılı denir.

Bu yöntemin sakıncaları vardır:

- **Algoritmayı değil, gerçekleştirmeyi karşılaştırır.** Kodlama programcının stiline sıkı biçimde bağlıdır. Çok etkin yazılabilecek bir kodu bir programcı çok kötü yazabilir.
- **Bilgisayar sonucu etkiler.** Birinci algoritma çalışırken hiçbir başka süreç ve ağ trafiği olmayabilir. İkincisi çalışırken pek çok süreç aynı anda çalışıyor, bellekte sayfalama (paging) yapılıyor, çok fazla disk erişimi oluyor ve ağ trafiği yüzünden zaman kaybediliyor olabilir. O an çalışan her şey sonucu etkiler.
- **Derleyici optimizasyonları sonucu etkiler.** Birinde optimizasyon açık, birinde kapalı olabilir. Farklı işletim sistemleri farklı optimizasyonlar yapabilir.

Bu yüzden deneysel karşılaştırma adil (fair) ve makul bir kıyas yöntemi değildir; çok fazla değişkenden etkilenir. Çok karmaşık durumlarda ya da elinizde yalnız belli bir veri varsa, mümkün olduğunca eşit şartlarda karşılaştırma yapılabilir. Çok sayıda deney yapıp ortalama almak da tekil kötü örneklerin etkisini azaltır. Ama yine de iyi bir yöntem değildir.

> **Hocanın vurgusu:** Çalışma süresini kronometreyle ölçmek algoritmayı değil gerçekleştirmeyi karşılaştırır. Bu yüzden teorik analiz tercih edilir.

7.2 Teorik Analiz

Teorik analizde bir algoritmanın başarımı gerçekleştirmeden, çalışılan ortamdan ve veriden bağımsız olarak, matematiksel yöntemlerle değerlendirilir. Temel yaklaşım şudur:

- Önce algoritmanın içindeki önemli işlemlerin sayısı, yani **maliyet noktaları** belirlenir.
- Sonra girdi boyutu büyüdüğünde bu sayının ne olduğunu göstermek için **büyüme fonksiyonları** (growth functions) kullanılır. Algoritmanın etkinliği bu fonksiyonların mertebesiyle ifade edilir.

8. İşlem Maliyetlerinin Sayılması

Bir algoritmada yaptığımız her işlemin bir maliyeti vardır. Atamanın bir maliyeti, aritmetik işlemin bir maliyeti vardır; toplama ile çarpmanın maliyeti farklıdır. Bu maliyet işlemcinin komut kümesine (instruction set), işlemin kaç saat çevrimi (clock cycle) sürdüğüne ve hangi yazmaçlardan (register) ne kadar okuma ve yazma gerektiğine göre değişir. Bir karar yapısındaki karşılaştırma ve dallanma ayrı bir maliyettir. Bir döngünün kontrolü de ayrı bir maliyettir. Özetle bunların hepsi bir işlemci için **sabit** ama ölçülebilir bir süre gerektirir.

8.1 Ardışık Komutlar

```
count = count + 1;   /* maliyet c1 */
sum = sum + count;   /* maliyet c2 */
```

`count = count + 1` şunu yapar: Değişkeni al, değerine sabit bir değer ekle, sonucu aynı değişkenin yerine yaz. İşlemci açısından birkaç iş yapılır, ama toplam süre sabittir. Buna c_1 diyelim. `sum = sum + count` ise bir değişkeni başka bir değişkenle toplar. Burada sabit yerine değişken okunduğu için maliyet farklı olabilir. Buna c_2 diyelim. Toplam maliyet:

$$T = c_1 + c_2$$

8.2 if Yapısı

Hocanın akış örneğinde bir `if` vardı: `n < 0` kontrolü yapılıyor. Doğruysa `a = -n`, yanlışsa `a = n` çalışıyor (mutlak değer hesabı). Karşılaştırmanın maliyeti c_1'dir ve her zaman bir kez yapılır. Doğru kolunun maliyeti c_2, yanlış kolunun maliyeti c_3'tür. İkisi aynı anda çalışamaz; ya biri ya öbürü çalışır. Hangisinin daha büyük olduğunu bilmiyorsak büyük olanı alırız:

$$T = c_1 + c_k, \qquad c_k = \max(c_2, c_3)$$

8.3 Tek Döngü

Satır	Maliyet	Tekrar
`i = 1;`	c_1	1
`sum = 0;`	c_2	1
`while (i <= n)`	c_3	$n+1$
`i = i + 1;`	c_4	n
`sum = sum + i;`	c_5	n

Döngü koşulu n kez doğru çıkar ve bir kez daha (çıkarken) kontrol edilir; bu yüzden $n+1$ kez çalışır. Gövdedeki iki satır n kez çalışır. Toplam:

$$T(n) = c_1 + c_2 + (n+1)c_3 + nc_4 + nc_5 = n(c_3 + c_4 + c_5) + (c_1 + c_2 + c_3)$$

$c_3 + c_4 + c_5$ ne olursa olsun bir sabittir. Yani $T(n) = an + b$ biçimindedir. Bu döngünün işletim süresi girdi boyutu n ile **doğru orantılıdır**. $n = 10$ ise sabit çarpı 10, $n = 1000$ ise sabit çarpı 1000, $n = 1$ milyon ise sabit çarpı 1 milyon kadar sürer.

8.4 İç İçe Döngüler

Döngü sayısı ikiye çıkarıldığında:

Satır	Maliyet	Tekrar
`i = 1;`	c_1	1
`sum = 0;`	c_2	1
`while (i <= n)`	c_3	$n+1$
`j = 1;`	c_4	n
`while (j <= n)`	c_5	$n(n+1)$
`sum = sum + i;`	c_6	$n \cdot n$
`j = j + 1;`	c_7	$n \cdot n$
`i = i + 1;`	c_8	n

İç döngünün kontrolü, iç döngü her seferinde $n+1$ kez kontrol edildiği ve dış döngü n kez döndüğü için $n(n+1)$ kez çalışır. İç gövde $n \cdot n$ kez çalışır. İç döngü bittikten sonra, hâlâ dış döngünün içinde olan `i = i + 1` satırı n kez çalışır. Toplam:

$$T(n) = c_1 + c_2 + (n+1)c_3 + nc_4 + n(n+1)c_5 + n^2c_6 + n^2c_7 + nc_8$$

Burada $n^2(c_5 + c_6 + c_7)$ biçiminde bir bileşen vardır. Yani $T(n) = an^2 + bn + c$ biçimindedir ve n^2 ile mertebe olarak orantılıdır.

8.5 Genel Kurallar

Hoca maliyet hesabını şu kurallarla özetledi:

- **Döngü:** Bir döngünün işletim süresi en fazla, döngünün tekrar (iterasyon) sayısı ile döngü içindeki komutların toplam işletim süresinin çarpımıdır.
- **İç içe döngü:** En içteki döngü bloğundaki komutların toplam işletim süresi, bütün döngülerin tekrar sayılarının çarpımıyla çarpılır. İç içe üç döngü olsaydı ve hepsi n'e bağlı olsaydı $n \cdot n \cdot n$, yani n^3 mertebesinde süre gerekirdi.
- **Ardışık komutlar:** Maliyetler toplanır: $c_1 + c_2 + c_8 + c_9$ gibi. Toplam bir sabittir.
- **if yapısı:** Doğru ve yanlış kollarından hangisi daha çok zaman gerektiriyorsa o alınır (maksimum). Üzerine karşılaştırmanın maliyeti eklenir. Böylece olabilecek en uzun işletim süresi elde edilir; algoritma bundan daha uzun süre çalışamaz.

9. Büyüme Oranı

Büyüme oranı (growth rate) şu soruyu cevaplar: Problemin boyutu, yani girdi sayısı arttıkça, problemi çözmek için gereken toplam süre ne kadar ve hangi oranda artıyor?

Bir algoritmanın zaman gereksinimi problemin boyutunun bir fonksiyonu olarak ölçülür. n girdi varsa zamanı $T(n)$ gibi n'ye bağlı bir fonksiyonla ifade ederiz. Boyutun ne olduğu uygulamaya bağlıdır. Örneğin bir sayı dizisini sıralıyorsak boyut eleman sayısıdır. Bir kullanıcı listesinde belli bir kişiyi arıyorsak boyut listedeki kullanıcı sayısıdır.

Hocanın örneği: A algoritması problemi $n^2/5$ birim zamanda, B algoritması $5n$ birim zamanda çözüyor. Anlamamız gereken en önemli nokta, zaman gereksiniminin n'ye bağlı olarak **hangi oranda** büyüdüğüdür. A'nın zaman ihtiyacı n^2 ile orantılı, B'ninki n ile, yani doğrusal olarak büyür. Aynı problemi çözen algoritmaların başarımı büyüme oranlarına bakılarak karşılaştırılır.

> **Ders sonrası not:** $n^2/5$ ile $5n$ fonksiyonları $n^2/5 = 5n$ eşitliğinden $n = 25$'te kesişir. Hocanın sonra gösterdiği grafikteki $n = 1000$ başa baş noktası, bu iki fonksiyona değil, aşağıdaki ayrı sayısal örneğe aittir.

9.1 Grafik ve Sayısal Örnek

Hoca bir çalışma zamanı grafiği çizdi. Doğrusal algoritma kesikli çizgiyle gösterilmişti ve doğrusal artıyordu. Diğeri karesel artıyordu. $n = 1000$'e kadar, örneğin $n = 500$'de, doğrusal algoritmanın çalışma süresi daha yüksekti; karesel olan daha hızlı sonuç üretiyordu. $n = 1000$'de ikisi aynı sürede sonuç üretti. Sonra, örneğin $n = 1500$'de, karesel algoritma çok daha fazla zamana ihtiyaç duydu. n büyüdükçe karesel eğri "göremediğimiz yerlere" çıkar.

Sayısal örnek: Doğrusal algoritma $n = 10$ için 10^{-2} s, $n = 100$ için 10^{-1} s, $n = 1000$ için 1 s sürüyor. Karesel algoritma $n = 1000$'de onunla başa baş gelip 1 s sürüyor. $n = 1000$ öncesi küçük değerler olduğu için ihmal edilebilir. n'yi 10 ile çarptıkça doğrusal sürenin sonuna bir sıfır, karesel sürenin sonuna iki sıfır eklenir:

n	Doğrusal	Karesel
1000	1 s	1 s
10 000	10 s	100 s
100 000	100 s	10 000 s $\approx$ 2,78 saat

$10\,000/3600 \approx 2{,}78$ saat (python ile doğrulandı). Başlarda çok hızlı görünen bir algoritma, belli bir eşikten sonra, hem de n'nin çok büyük olmayan bir değerinden sonra çok yavaşlayabilir.

> **Ders sonrası not:** Hoca $n = 100\,000$ için doğrusal algoritmanın "2 saniyenin altında" sonuç ürettiğini söyledi. Tabloya göre doğrusu 100 saniyedir, yani 2 dakikanın altındadır.

Büyüme oranı daha büyük olan ikinci (karesel) algoritmadır. Daha etkin çalışan birincisidir, çünkü doğrusal artar. Hoca karesel artışı burada ve birkaç yerde daha "üstel" diye andı.

Ders sonrası not: n^2 büyümesi üstel değil, karesel (quadratic), yani polinomsal büyümedir. Üstel büyüme 2^n gibi, n'nin üste çıktığı fonksiyonlardır. Hoca da aynı cümlede "kuadratik dediğimiz bir artış" diye düzeltti.

10. Yaygın Büyüme Oranları ve Döngü Örnekleri

10.1 Büyüme Oranları Sıralaması

Hoca sık kullanılan büyüme oranlarını iyiden kötüye sıraladı:

Gösterim	Ad	Açıklama
$O(1)$	Sabit (constant)	Girdi boyutundan bağımsız; sabit sayıda işlem
$O(\log n)$	Logaritmik	İkiye bölerek ilerleyen algoritmalar, örneğin ikili arama
$O(\log^2 n)$	Log-kare	Logaritmikten bir kötü
$O(n)$	Doğrusal (linear)	Boyutla paralel artış
$O(n \log n)$	Doğrusal logaritmik (linearithmic)	Doğrusaldan $\log n$ kat fazla
$O(n^2)$	Karesel (quadratic)	10 kat büyüyen girdi için 100 kat süre
$O(n^3)$	Kübik	Üçüncü dereceden artış
$O(2^n)$	Üstel (exponential)	Örneğin özyinelemeli Fibonacci
$O(n!)$	Faktöriyel	Bütün permütasyonları denemek
$O(n^n)$		Faktöriyelden de kötü

Açıklamalar:

- **Sabit:** En iyi ve en etkin olanıdır. Girdi 10 elemanlı da olsa 10 milyon elemanlı da olsa aynı sayıda işlem yapılır. Ama her zaman mümkün değildir; yalnız belirli problemlerde sabit sayıda işlemle çözüm vardır.
- **Logaritmik:** Hoca bu derste log yazdığında **her zaman 2 tabanını** kastettiğini söyledi. Sebebi, özünde ikili (binary) çalışıyor olmamızdır.
- n^4, n^5, n^6 diye üsler artırılabilir. n^4, n^2'nin karesi olduğu için karesel mertebenin karesi gibi de düşünülebilir. Hoca n^3'ten sonrasını konuşmaya fazla gerek olmadığını söyledi.
- **Üstel:** Artık polinom zamanda çözüm üretemeyeceğimiz mertebedir. Zaman gereksinimi algoritmanın kullanışlı olmasını engelleyecek kadar hızlı artar. Özyinelemeli çağrılarla Fibonacci hesabı böyledir; gelecek hafta görülecek.
- **Faktöriyel:** $n \cdot (n-1) \cdot (n-2) \cdots 1$. Bütün olası kombinasyon ve permütasyonların oluşturulup denenmesi, yani kapsamlı arama (exhaustive search) böyledir. Brute force'un kötülüğü burada ortaya çıkar. Yalnız problemin çok küçük örnekleri için mantıklı sürede çözülebilir.

  Hocanın vurgusu: Logaritmalar bu derste hep 2 tabanındadır. $\log_2 n$ yazılmasa da $\log n$ her zaman 2 tabanında okunmalıdır.

Hoca $n!$'den daha kötü ne olabileceğini sordu. Cevap: n^n. Bu soru "Örnek Sorular" bölümünde.

Hoca bu fonksiyonların grafiğini de gösterdi. Önce 100'e 100'lük milimetrik kâğıt üzerinde: $n!$ daha $n = 5$-6'da yüzlere vurduğu için grafik okunmuyordu. Sonra girdi ve çıktı eksenleri ikinin katları olarak ölçeklenmiş bir grafik gösterildi. Bu ölçek yüzünden doğrusal fonksiyon düz çizgi gibi görünmüyordu. Kural şudur: Büyüme eğrisi yatay eksene ne kadar yakınsa o kadar iyi, dikey eksene ne kadar yakınsa o kadar kötüdür. Aynı problemi $n \log n$ ile çözen algoritma, n^2 ile çözen algoritmaya tercih edilir.

10.2 Döngü Örnekleri

Hoca slaytta $n = 1000$ alınmış döngülerle büyüme oranlarını gösterdi.

Doğrusal:

```
for (i = 0; i < n; i++) { /* sabit iş */ }      /* n adım: O(n) */
```

0'dan n'ye birer birer ilerler; $f(n) = n$.

İkişer artan döngü:

```
for (i = 0; i < n; i += 2) { /* sabit iş */ }   /* n/2 adım: O(n) */
```

$n = 1000$ için i değerleri 0, 2, 4, ..., 998'dir; 500 değer alır (python ile doğrulandı). Yani $n/2$ adımdır, ama yine n ile orantılı ve doğrusaldır.

Çarparak ilerleyen döngü:

```
for (i = 1; i < n; i *= 2) { /* sabit iş */ }   /* log n adım: O(log n) */
```

$n = 1000$ için i değerleri 1, 2, 4, 8, 16, 32, 64, 128, 256, 512'dir. Toplam 10 adım, yani yaklaşık $\log_2 n$ adım. Logaritmik büyüme oranına sahiptir.

İç içe iki döngü:

```
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++) { /* sabit iş */ }  /* n * n: O(n^2) */
```

Dış döngü n, iç döngü n kez döner; içerideki işler $n \cdot n$ kez yapılır. Karesel büyüme.

Bağımlı karesel döngü:

```
for (i = 0; i < n; i++)
    for (j = 0; j < i; j++) { /* sabit iş */ }  /* O(n^2) */
```

İç döngü i'ye bağlıdır. Hoca bunu bir üçgen olarak gösterdi ve toplam işlem sayısının $n(n+1)/2 = (n^2+n)/2$ olduğunu söyledi. Katsayı $1/2$ olsa da n^2 mertebesinde bir bileşen vardır. Buna bağımlı karesel (dependent quadratic) denir.

> **Ders sonrası not:** `j < i` koşuluyla ve i, 0'dan $n-1$'e giderken iç gövde tam olarak $0 + 1 + \cdots + (n-1) = n(n-1)/2$ kez çalışır ($n = 10$ için 45, python ile sayıldı). Hocanın verdiği $n(n+1)/2$, iç döngü $j \leq i$ olsaydı geçerli olurdu. İki durumda da sonuç $O(n^2)$'dir.

Doğrusal logaritmik:

```
for (i = 0; i < n; i++)
    for (j = 1; j < n; j *= 2) { /* sabit iş */ } /* n * log n: O(n log n) */
```

Hoca iç döngünün özellikle 1'den başladığına dikkat çekti; 0'dan başlasaydı çarparak hiç ilerleyemez, 0'da takılı kalırdı. İç döngü logaritmik, dış döngü doğrusaldır. Toplam $n \log n$.

11. Big O Gösterimi

11.1 Sezgisel Tanım

Hoca "şöyle bir nefes alın, bunu iyice oturtmamız gerekiyor" diyerek başladı. Bir algoritmanın büyüme fonksiyonu $f(n)$, başka bir $g(n)$ fonksiyonu ile orantılı zaman gerektiriyorsa (örneğin $5g(n)$ ya da $\frac{1}{3}g(n)$ gibi), $f(n)$, $g(n)$ ile aynı büyüme seviyesindedir. Ya da $g(n)$'nin ait olduğu büyüme oranı kümesindedir. Bunu ifade etmek için Big O gösterimini kullanırız ve $O(g(n))$ yazarız. $g(n)$ önceden tanımlı bir büyüme oranı fonksiyonudur.

Neden "O"? Bir mertebeyi ifade etmek için O harfi kullanılır. Hoca harfin sıfırla karışmaması için, İngilizcede okunuşa uygun olarak "Big Oh" diye "h" eklenerek yazılmasının tercih edildiğini, böylece ileride görülecek teta gibi gösterimlerle de karışmayacağını söyledi.

Örnek: $g(n) = n^2$ olsun. Önceki slaytlardaki karesel algoritma $f(n) = n(n+1)/2$ zaman gerektiriyor olsun:

$$f(n) = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

Bu ifade n^2 mertebesinde büyür, yani n^2 ile orantılıdır. O hâlde $f(n) = O(n^2)$ yazılabilir: Algoritmanın büyüme oranı n^2 mertebesindedir. $g(n) = n$ ile orantılı olsaydı $O(n)$ mertebesinde derdik.

Sık kullanılan büyüme oranı fonksiyonlarının Big O karşılıkları, Bölüm 10.1'deki tablodaki gibidir. Sabit büyüme c yerine 1 yazılarak $O(1)$ ile gösterilir. $O(1)$'in de bir maliyeti vardır, ama bu maliyet n'ye bağlı değildir; problemin boyutundan bağımsızdır.

11.2 Sayısal Karşılaştırma

Hoca $n = 10\,000$ için farklı mertebelerin işlem sayılarını karşılaştırdı:

- $O(\log n)$: $\log_2 10\,000 \approx 13{,}3$, yani yaklaşık 13 işlem. Modern bir işlemcide büyük olasılıkla mikrosaniyeler mertebesinde biter.
- $O(n)$: 10 000 iterasyon.
- $O(n \log n)$: Hoca $\log n$'yi 14'e yuvarlayıp yaklaşık 140 000 iterasyon dedi. Tam değer $10\,000 \cdot 13{,}29 \approx 132\,900$'dür (python).
- $O(n^2)$: $10\,000 \cdot 10\,000 = 10^8$ işlem.
- $O(n^k)$: $10\,000^k$ işlem.
- $O(2^n)$: $2^{10\,000}$ işlem. Hoca buna "felaket" dedi.
- $O(n!)$: $10\,000!$ işlem. Ömrümüzün yetmeyeceği bir çalışma süresi.

  Ders sonrası not: Hoca bu işlem sayılarına süre de biçti: doğrusal için "saniyeler", $n \log n$ için de benzer, n^2 için "dakikalar", n^k için "saatler". Bu süreler abartılıdır. Saniyede büyük olasılıkla 10^9 mertebesinde basit işlem yapan bir işlemcide 10^4 işlem mikrosaniyeler, 10^8 işlem yaklaşık 0,1 saniye sürer. Burada önemli olan mutlak süreler değil, mertebelerin sıralamasıdır.

Aynı karşılaştırma farklı n değerleri için tabloyla gösterildi. Hoca slayttaki n sütununda da bir yanlış yazım olduğunu söyleyip düzeltti. Tablodaki logaritma değerleri aşağı yuvarlanmıştır (python ile kontrol edildi; örneğin $\log_2 10^6 \approx 19{,}93$):

n	$\log n$	$n \log n$	n^2	n^3
10	3	30	10^2	10^3
100	6	600	10^4	10^6
1000	9	9000	10^6	10^9
10 000	13	130 000	10^8	10^{12}
100 000	16	$1{,}6 \cdot 10^6$	10^{10}	10^{15}
1 000 000	19	$\approx 10^7$	10^{12}	10^{18}

$O(1)$ hep sabittir; girdi boyutundan etkilenmez. Logaritmikte 1 milyon girdide bile en fazla 20. adımda sonuç bulunur. 2^n için $n = 100$'de yaklaşık 10^{30}, $n = 1000$'de yaklaşık 10^{301} işlem gerekir ($2^{100} \approx 1{,}27 \cdot 10^{30}$, $2^{1000} \approx 1{,}07 \cdot 10^{301}$; python). Bu kadar işlem için günler, aylar, belki asırlar beklemek gerekir. Böyle bir algoritma teoride problemi çözer, ama pratikte çözemez; uygulanabilir (feasible) bir çözüm değildir.

11.3 Biçimsel Tanım

Hoca sonra algoritmanın seviyesinin (mertebesinin) nasıl bulunduğunu biçimsel olarak tanımladı. Örnek algoritma n boyutlu girdiyle çalışıp sonucu üretmek için

$$f(n) = n^2 - 3n + 10$$

saniye gerektiriyor. Bu fonksiyon, daha önceki c_2, c_3 gibi maliyetler çarpılıp toplanarak elde edilmiş olabilir.

Tanım (Big O): $f(n) = O(g(n))$'dir, eğer öyle **pozitif** k ve n_0 sabitleri bulunabiliyorsa ki her $n \geq n_0$ için

$$0 \leq f(n) \leq k \cdot g(n)$$

sağlanır.

Örnekte $g(n) = n^2$ alınır, yani en yüksek mertebeli terim. $k = 3$ ve $n_0 = 2$ seçilirse her $n \geq 2$ için

$$3n^2 \geq n^2 - 3n + 10$$

doğrudur. Kontrol: Eşitsizlik $2n^2 + 3n - 10 \geq 0$ demektir. $n = 2$'de $8 + 6 - 10 = 4 \geq 0$, sonra da sol taraf artar. $n = 1$'de ise $2 + 3 - 10 = -5 < 0$ olduğu için $n_0 = 1$ seçilemez (ara adım eklendi; python ile $n = 1..39$ için kontrol edildi). Hoca k'nin tam sayı olmak zorunda olmadığını da belirtti; yalnız pozitif olmalıdır.

Bu, şu anlama gelir: Algoritma $n \geq n_0$ olduğunda kn^2 birim zamandan fazlasına asla ihtiyaç duymaz. Yani algoritmanın seviyesi en kötü durumda n^2 mertebesindedir; $f(n) = O(n^2)$.

Grafikte: $n_0 = 2$ noktasından dikey bir çizgi çekilir. Bu çizginin solundaki bölge önemsizdir; hoca "I don't care about this area" dedi. Orada $f(n)$, $kg(n)$'nin üstüne çıkabilir. Ama çizginin sağında, çok ilerilerde bile, $kg(n)$ hep yukarıda, $f(n)$ hep aşağıda kalır. $f(n)$ en fazla ona yaklaşabilir, ama asla geçemez. Böylece $f(n)$, n^2 mertebesiyle **üstten asimptotik olarak sınırlanmış** olur. Buna algoritmanın mertebesi ya da seviyesi denir.

> **Hocanın vurgusu:** Big O gösterimi algoritmayı üstten sınırlar. "Bu algoritma bundan daha kötü olamaz" demektir.

12. Big Omega ve Big Theta

Big O ile hep "en fazla bu kadar zamana ihtiyaç duyar" dedik, yani üstten sınırladık. Aynı şeyi alttan da yapabiliriz.

Big Omega: Bir algoritma, $n \geq n_0$ boyutundaki bütün problemleri çözmek için en az $kg(n)$ birim zamana ihtiyaç duyuyorsa, yani öyle pozitif k ve n_0 sabitleri bulunabiliyorsa ki her $n \geq n_0$ için

$$f(n) \geq k \cdot g(n)$$

sağlanıyorsa, algoritma $\Omega(g(n))$ seviyesindedir. Bu durumda g hep aşağıda, f hep yukarıda kalır; f asla g'nin altına düşemez. Fonksiyon alttan asimptotik olarak sınırlanmış olur. Hoca bunun bu senenin konusu olmadığını, ama madem Big O'dan bahsettik, neden alttan da sınırlamayalım diye gösterdiğini söyledi.

Ardından bir öğrenci sordu:

Soru: Üstten de sınırlayabiliyoruz, alttan da sınırlayabiliyoruz. Fonksiyonu hem üstten hem alttan aynı anda sınırlayabilir miyiz? **Cevap:** Evet. O zaman iki sabite ihtiyaç duyulur. Bu Big Theta'dır.

Big Theta: Öyle pozitif k_1, k_2 ve n_0 sabitleri bulunabiliyorsa ki her $n \geq n_0$ için

$$k_1 g(n) \leq f(n) \leq k_2 g(n)$$

sağlanıyorsa, $f(n) = \Theta(g(n))$'dir. Fonksiyon hem üstten hem alttan aynı anda sınırlanır. Hoca kitapta bu sabitlerin c_1, c_2 diye yazıldığını belirtti.

Hoca üçünü yan yana bir slaytta gösterdi:

- **Big O:** Fonksiyon n_0'dan sonra $g(n)$'nin hep altında kalır; en fazla dibine kadar gelebilir, asla geçemez. Üst asimptot tanımlanır.
- **Big Omega:** Fonksiyon asla $g(n)$'nin altına inemez. Alt asimptot tanımlanır.
- **Big Theta:** Fonksiyon $c_1 g(n)$ ile $c_2 g(n)$ arasında kalır. İki asimptot aynı anda tanımlanır.

Hoca ayrıca bu üç gösterimi durumlarla eşleştirdi: Big O'yu en kötü durum, Big Omega'yı "en iyi durum analizi", Big Theta'yı "ortalama durum" ile ilişkilendirdi.

> **Ders sonrası not:** Big O, Big Omega ve Big Theta bir fonksiyonun üst, alt ve iki taraflı **sınırlarıdır**. En kötü, en iyi ve ortalama durum ise **hangi girdinin** incelendiğini belirtir. Bunlar farklı kavramlardır. Her durumun maliyeti O, Ω ya da Θ ile ifade edilebilir. Örneğin insertion sort'un en kötü durumu $\Theta(n^2)$'dir, en iyi durumu $\Theta(n)$'dir.
>
> **Hocanın vurgusu:** Bu ders kapsamında algoritmalar yalnız üstten, yani Big O ile sınırlanacak. "En kötü durumda bundan kötü olamam, bu algoritmadaki en kötü mertebe budur" çıkarımı yapılacak.

Hoca son olarak, tanımlardaki eşitlik kaldırılıp kesin eşitsizlik kullanılırsa küçük o ve küçük omega gösterimlerinin elde edildiğini, küçük teta diye bir şey olmadığını söyledi. Bu gösterimlerde sınır fonksiyonuna "değmeme" durumu ifade edilir.

> **Ders sonrası not:** Küçük o'nun tanımı yalnız eşitliği kaldırmak değildir. $f(n) = o(g(n))$, **her** pozitif k için (yalnız bir k için değil) $f(n) < kg(n)$'yi sağlayan bir n_0 bulunabilmesi demektir. Örneğin $n = o(n^2)$'dir ama $n^2 \neq o(n^2)$'dir. Küçük omega da benzer biçimde tanımlanır.

13. Big O'nun Özellikleri ve Örnekler

13.1 Özellikler

Big O'nun işi kolaylaştıran bazı özellikleri vardır.

1. Düşük dereceli terimler atılır. Bir fonksiyon $n^3 + 4n^2 + 3n$ bulunduysa, n^3 varken kimse n^2 ya da n terimine takılmaz. Fonksiyonu baskın biçimde üstten sınırlayan terim n^3'tür:

$$n^3 + 4n^2 + 3n = O(n^3)$$

Büyüme fonksiyonu olarak yalnız en yüksek dereceli terimi kullanmak yeterlidir.

2. Sabit çarpanlar atılır. Bir algoritma $5n^3$ mertebesindeyse, "n^3 varken 5'e, 3'e, 10'a kim takar?" Sabit atılır: $5n^3 = O(n^3)$.

3. Başka bir değişkene bağlı çarpan atılmaz. İkinci bir girdi varsa, örneğin m ya da k, bu çarpan da değişken olduğu için atılamaz; mn^3 ya da kn^3 yazılır. Çarpan n olsaydı da atılamazdı, çünkü o zaman ifade n^4 olurdu. Hoca graf algoritmalarında hem kenarların hem düğümlerin karmaşıklık yarattığını, bu yüzden V ve E içeren ifadeler göreceğimizi söyledi.

4. Büyüme fonksiyonları birleştirilebilir.

$$O(f(n)) + O(g(n)) = O(f(n) + g(n))$$

Örneğin $O(n^3) + O(4n^2) = O(n^3 + 4n^2)$. Yukarıdaki özellikler sayesinde bu $O(n^3)$ olur. Çarpmada da aynı kural geçerlidir; iç içe yapılarda çarpılınca üsler artabileceği için dikkat etmek gerekir.

> **Hocanın vurgusu:** Big O'da düşük dereceli terimler ve sabit çarpanlar atılır. Başka bir değişkene bağlı çarpan (örneğin m, graflarda V ve E) atılmaz.

13.2 Örnekler

Örnek 1: Tek döngü. Bölüm 8.3'teki kod. Döngü koşulu $n+1$ kez, gövdedeki iki satır n'er kez çalışır:

$$T(n) = c_1 + c_2 + (n+1)c_3 + nc_4 + nc_5$$

n'li terimler paranteze alınıp a, sabitler b denirse (n^1 ve n^0 terimleri) $T(n) = an + b$ olur. Düşük terimi ve sabiti atınca $O(n)$ kalır. Bu kod parçası en kötü durumda doğrusal büyüme oranına sahiptir.

Örnek 2: İç içe iki döngü. Bölüm 8.4'teki kod:

$$T(n) = c_1 + c_2 + (n+1)c_3 + nc_4 + n(n+1)c_5 + n^2c_6 + n^2c_7 + nc_8$$

n^2'li terimler paranteze alınıp a, n'li terimler b, sabitler c denirse $T(n) = an^2 + bn + c$ olur. Küçük terimler ve a atılır: $O(n^2)$.

Örnek 3: İç içe üç döngü.

```
for (i = 1; i <= n; i++)
    for (j = 1; j <= i; j++)
        for (k = 1; k <= j; k++)
            x = x + 1;          /* maliyet c1 */
```

Toplam işlem sayısı

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{i} \sum_{k=1}^{j} c_1$$

biçimindedir. Dış döngü n mertebesindedir; iç döngüler bağımlıdır ve n'den azdır ("üçgen"). Ama her biri n mertebesindedir. Katsayılar ve düşük dereceli terimler atılınca $O(n^3)$ kalır. Döngülerden biri logaritmik olsaydı iş değişirdi; örneğin $O(n^2 \log n)$ olabilirdi.

> **Ders sonrası not:** Hoca bu toplamı "$\frac{n(n+1)}{2} \cdot \frac{n(n+1)}{2}$" diye yazmaya başladı. Doğrusu $\sum_{i=1}^{n} \frac{i(i+1)}{2} = \frac{n(n+1)(n+2)}{6}$'dır (python ile $n = 10$ için 220 sayıldı). Hocanın ifadesi n^4 mertebesine çıkardı. Doğru toplam da $n^3/6$ baskın terimiyle $O(n^3)$'tür.

14. En Kötü, En İyi ve Ortalama Durum

Bir algoritma aynı boyuttaki farklı girdiler için farklı sürelere ihtiyaç duyabilir. Hoca bunu sıralı bir diziyle gösterdi:

$$2,\ 3,\ 6,\ 7,\ 9,\ 10,\ 11 \qquad (n = 7)$$

En kötü durum (worst case): $x = 15$ aranıyor. Doğrusal arama baştan başlar: 15'e eşit mi? Hayır, hayır, hayır... Sona kadar gelinir; n elemanın hepsiyle karşılaştırılır ve bulunamaz. Bu, yaşanabilecek en kötü durumdur. En kötü durum analizi, algoritmanın n boyutlu problemi çözmek için ihtiyaç duyduğu **en uzun** süreyi verir; karmaşıklığın üst sınırıdır. Bu ders kapsamında hep en kötü durum davranışı incelenecek.

En iyi durum (best case): $x = 2$ aranıyor. İlk elemanda bulunur. Tek bir karşılaştırma, yani sabit sayıda işlem: $O(1)$. En iyi durum analizi n boyutlu problemi çözmek için gereken **en az** süreyi verir. Çok kullanışlı değildir, ama "bundan aşağı düşemezsin; bir atışta bulsan bile bu kadar işlemi yapmak zorundasın" bilgisini verir.

Ortalama durum (average case): n boyutlu problemi çözmek için gereken ortalama süredir. Belirlemesi çoğunlukla zordur; olası veri durumları ve dağılımları değerlendirilir. Hocanın

tarif ettiği yol: Bu 7 elemanlı dizide örneğin 210 sayı üretilip aranır, bulunan adım sayıları toplanıp 210'a bölünür. İçinde en iyi durum da en kötü durum da olur, çoğunlukla da ortadaki durumlar. Yeterince çok tekrar edilip ortalaması alınınca "bu algoritma ortalamada şu kadar süre gerektirir" çıkarımı yapılır. Ortalama durumun da analiz yöntemleri vardır, ama en kötü durum analizi genellikle daha çok kullanılır.

> **Hocanın vurgusu:** Bu derste değerlendirmeler hep en kötü durum üzerinden ve Big O ile yapılacak: "Bu algoritmayla bundan daha kötü olamayız."

15. Sıralı Arama ve İkili Arama

Hoca tahtada olsaydı bunların akışlarını çizeceğini söyledi ve öğrencilerden bu algoritmaları mutlaka akış diyagramına dökmelerini istedi. Çoğu geçen sene görülmüştü, ama kontrollere (eşitlik, küçük, büyük, küçük eşit) dikkat ederek akışları tekrar tasarlamak faydalıdır.

15.1 Sıralı Arama

Hoca slayttaki kodun "kötü bir kodlama örneği" olduğunu söyledi: Döngü gövdesi boştur, bütün iş koşulda yapılır. $i < n$ olduğu ve eleman aranan değere eşit olmadığı sürece bakmaya devam edilir. Sona gelindiyse bulunamamıştır; gelinmediyse i konumunda bulunmuştur.

```
int sequentialSearch(int a[], int size, int x) {
    int i;
    for (i = 0; i < size && a[i] != x; i++);
    if (i == size)
        return -1;
    return i;
}
```

Kod gcc ile derlendi; 2, 3, 6, 7, 9, 10, 11 dizisinde 2 için 0, 11 için 6, 15 için -1 döndürdü.

- **Bulunamazsa** ($x = 15$): Bütün n elemanla karşılaştırılır: $O(n)$.
- **En iyi durum:** Aranan ilk elemandır ($x = 2$). Tek işlem: $O(1)$.
- **En kötü durum:** Aranan son elemandır ($x = 11$). 11 bulunana kadar 7 elemanla karşılaştırılır: $O(n)$.
- **Ortalama durum:** 1, 2, 3, ..., 7 karşılaştırma yapılan aramaların sayılarının eşit olması beklenir. Toplam n'ye bölününce ortalamada yaklaşık $n/2$ adımda sonuca varılır. Big O'da $1/2$ sabitinin önemi yoktur; o da silinir ve $O(n)$ kalır.

  Ders sonrası not: Slayttaki ortalama formülünde pay $\sum_{i=1}^{n} n$ biçiminde görünüyordu. Doğrusu $\frac{1}{n}\sum_{i=1}^{n} i = \frac{n+1}{2}$'dir. Hoca sözlü olarak "toplam olasılık $1/2\,n$ gibi" dedi; kastedilen ortalama karşılaştırma sayısıdır.

15.2 İkili Arama

Dizi zaten sıralı olduğu için ikili arama da kullanılabilir. İkili arama, "ikiye böle böle ara" demektir.

```
int binarySearch(int a[], int size, int x) {
    int low = 0;
    int high = size - 1;
    int mid; /* aranan değer bulunursa mid konumu tutar */
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] < x)
            low = mid + 1;
        else if (a[mid] > x)
            high = mid - 1;
        else
            return mid;
    }
```

```
    return -1;
}
```

Kod gcc ile derlendi; aynı dizide 7 için 3, 11 için 6, 15 için -1 döndürdü.

Kodun mantığı: Sol ve sağ indeksler (`low`, `high`) tutulur, bir de orta konum (`mid`) hesaplanır. `low <= high` olduğu sürece, yani arama uzayının dışına çıkılmadığı sürece aramaya devam edilir. Ortadaki değer x'ten küçükse arama uzayı ortanın sağı olur; büyükse solu olur; eşitse bulunmuştur. Arama uzayı tükenirse "bulunamadı" diye çıkılır. Her adımda dizinin yarısından kurtulunur, sonra kalanın yarısından. Bu yüzden logaritmiktir.

En iyi durum: $x = 7$ aranıyor. Dizi 7 elemanlı; hoca $7/2 = 3{,}5$'i yukarı yuvarlayıp 4. elemandan başladığını söyledi (C kodunda 0 tabanlı mid $= (0 + 6)/2 = 3$, yani yine 4. eleman). İlk karşılaştırmada bulunur: $O(1)$.

En kötü durum: $x = 15$ aranıyor.

- Ortadaki eleman 7 ile karşılaştırılır. $7 < 15$, arama uzayı sağ yarı olur: 9, 10, 11.
- Bunun ortası 10'dur. $10 < 15$, arama uzayı yine sağ taraf olur: 11.
- Tek eleman 11 ile karşılaştırılır. $11 < 15$, sağa gidilmeye çalışılır ama dizinin sınırı aşıldığı için durulur.

En kötü durum, aranan değerin olmaması ($x = 15$) ya da son bakılan konumda olmasıdır ($x = 11$). Bu durumda $\log n + 1$ karşılaştırma yapılabilir. $+1$ sabiti atılır: En kötü durum $O(\log n)$'dir.

Ortalama durum: Karşılaştırma sayısının ortalamada $\log n$'den küçük, 1'den büyük olacağı bilinir. 7 elemanlı dizide:

- Ortadaki eleman (7) 1 adımda bulunur.
- 3 ve 10 elemanları 2 adımda bulunur.
- 2, 6, 9 ve 11 elemanları 3 adımda bulunur.

Konum sırasıyla adım sayıları 3, 2, 3, 1, 3, 2, 3'tür. Hoca bunları sırayla toplayarak 3, 5, 8, 9, 12, 14, 17 diye ilerledi:

$$1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 = 1 + 4 + 12 = 17 \text{ karşılaştırma}$$

$$\frac{17}{7} \approx 2{,}43 \ < \ \log_2 7 \approx 2{,}81$$

(python ile doğrulandı). Ortalama $\log n$'den biraz küçüktür, ama $\log n$ onu üstten sınırlar. Bu yüzden ortalamada da Big O'da $O(\log n)$ mertebesindedir.

Özet: Sıralı aramada en kötü ve ortalama durum $O(n)$, ikili aramada en kötü ve ortalama durum $O(\log n)$'dir. İkisinde de en iyi durum $O(1)$'dir.

16. Sıralama Algoritmalarının Karmaşıklığı

Hoca son olarak geçen seneden bilinen sıralama algoritmalarına hızlıca baktı.

16.1 Bubble Sort

Kabarcık sıralamasında (bubble sort) dizinin elemanları baştan sona doğru ikişer ikişer karşılaştırılır ve büyük olan sona taşınır. En büyük eleman bir kabarcık gibi "fıkır fıkır" dizinin sonuna çıkar. Sonra sıradaki en büyük eleman sondan ikinci sıraya, sonraki sondan üçüncü sıraya gider. Hoca akışı geçen sene çizdiklerini, bu sene slaytta anlatmak zorunda kaldığı için sözde koda çevirdiğini söyledi:

```
for i ← 0 to n − 2 do
    for j ← 0 to n − 2 − i do
        if A[j + 1] < A[j]
            tmp ← A[j]
```

```
            A[j] ← A[j + 1]
            A[j + 1] ← tmp
```

Hoca indislerin 0'dan $n-2$'ye yerine 1'den $n-1$'e de yazılabileceğini, bunun önemli olmadığını söyledi. Karmaşıklık: Dışta n mertebesinde bir döngü, içinde n mertebesinde ikinci bir döngü, onun içinde bir kontrol ve kontrolün içinde 3 sabit işlem vardır. Kabaca $n \cdot n \cdot c$ işlem yapılır. Sabit atılınca $O(n^2)$ kalır.

16.2 Selection Sort

Seçmeli sıralamada (selection sort) her turda, sıralama yönüne göre kalan elemanların en küçüğü (ya da en büyüğü) seçilir ve baştan başlayarak sıradaki yere yerleştirilir. Dış döngü n adım döner. İçinde sabit bir atama vardır: "en küçük elemanın indisi şudur". İç döngü i'den n'ye gider; bağımlıdır. İç döngünün içinde sabit sayıda işlem, iç döngünün dışında ise n mertebesinde sabit sayıda iş (yer değiştirme) vardır.

Hoca toplamın yaklaşık $\frac{1}{2}n^2 + \ldots$ biçiminde olduğunu söyledi. Düşük mertebeler ve katsayı atılınca $O(n^2)$ kalır. Karşılaştırma sayısı yaklaşık $n^2/2$ mertebesinde, yer değiştirme sayısı n mertebesindedir. Ama $T(n)$'deki n^2 terimi yüzünden selection sort da $O(n^2)$'dir.

16.3 Insertion Sort

Eklemeli sıralama (insertion sort), iskambil kâğıtlarını elde sıralamaya benzer. Sıradaki yeni eleman alınır ve önceki sıralı kısımla karşılaştırılarak olması gereken yere kaydırılır.

```c
void insertionSort(int A[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = A[i];
        /* A[i]'yi sıralı A[0..i-1] dizisine ekle */
        j = i - 1;
        while (j >= 0 && A[j] > key) {
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = key;
    }
}
```

Kod gcc ile derlendi ve bir örnek diziyi doğru sıraladı.

> **Ders sonrası not:** Slayttaki sözde kodda iç döngünün koşulu `j > 0` olarak görünüyordu (kısa nottaki slayt okumasına göre). Dizi 0'dan başladığı için doğrusu `j >= 0`'dır; aksi hâlde ilk eleman hiç karşılaştırılmaz. Yukarıdaki kodda düzeltildi.

Karmaşıklık:

- Dış döngü n elemanın üzerinden geçer: n mertebesi. İçinde sabit sayıda işlem vardır (yorum satırı atlanır).
- İçteki `while` "olduğu sürece" döner ve en kötü durumda n kez döner.
- Dış döngüde n, içteki döngüde de sabit çarpı n olduğu için en kötü durumda $c_2 n \cdot n$ mertebesinde işlem yapılır. Sabit atılınca $O(n^2)$.
- **En iyi durum:** Dizi zaten sıralıysa içteki `while` hiç çalışmaz. Tek bir doğrusal geçiş yapılır: $O(n)$.
- **Ortalama durum:** Hoca ortalamada $n^2/2$ karşılaştırma ve $n^2/2$ yer değiştirme yapıldığını söyledi. Ortalama durum da $O(n^2)$'dir.

  Ders sonrası not: Insertion sort'un ortalama durumda yaklaşık $n^2/4$ karşılaştırma ve $n^2/4$ kaydırma yaptığı bilinir; $n^2/2$ en kötü duruma aittir. Karmaşıklık yine $O(n^2)$'dir.

Bizim için en kötü durum önemli olduğu için insertion sort da $O(n^2)$ kabul edilir, ama en iyi durumu $O(n)$'dir.

Algoritma	En iyi	Ortalama	En kötü
Sıralı arama	$O(1)$	$O(n)$	$O(n)$
İkili arama	$O(1)$	$O(\log n)$	$O(\log n)$
Bubble sort (bu sözde kod)	$O(n^2)$	$O(n^2)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$

17. Logaritmik Karmaşıklığın Gücü

Hoca $\log n$'nin ne kadar etkili olduğunu tekrar göstermek için n'nin 2^4'ten başlayıp büyüyen değerlerine baktı:

- 16 eleman: 4 adım.
- 16 bin eleman: 14 adım ($\log_2 16\,000 \approx 13{,}97$).
- 1 milyon eleman: 20 adım ($\log_2 10^6 \approx 19{,}93$).
- 1 milyar eleman: 30 adım ($\log_2 10^9 \approx 29{,}9$).

(python ile doğrulandı.) Hoca bir örnek verdi: 80 milyon kişilik bir listede arama yapıyorsanız, istediğiniz kişiyi "22-23 adımda" bulursunuz ya da listede olmadığını söylersiniz. Bu yüzden $\log n$ çok iyi bir başarımdır ve özellikle arama algoritmalarında tercih sebebidir.

> **Ders sonrası not:** 80 milyon kişilik listede ikili arama $\lceil \log_2(8 \cdot 10^7) \rceil = \lceil 26{,}25 \rceil = 27$ adım sürer, "22-23" değil (python ile hesaplandı). Sonuç yine çok küçük bir sayıdır.

Örnek Sorular

Soru 1 (derste sözlü, 00:12:03): Slayttaki ağ görselinde işaretlenen iki düğüm (p ve q) birbirine bağlı mıdır?

> **Çözüm:** Öğrenciler "bağlıdır" dedi. Hoca bir yol çizerek bağlı olduklarını gösterdi. Bu problemde yalnız bağlı olup olmadıkları sorulur, hangi yolla bağlı oldukları sorulmaz.

Soru 2 (derste sözlü, 00:16:25 ve 00:18:38): Sırayla gelen `p-q` çiftleri için "bu ikisi zaten bağlı mı?" sorusunu nasıl çözebiliriz? Önceki bağlantıları tutmak için uygun veri yapısı ne olabilir?

> **Çözüm:** Öğrenciler matris tutup yol arama algoritması koşturmayı ve pointer'larla bağlı bir yapı kurmayı önerdi. Hocanın çözümü tek bir dizidir: `id[i]` her düğümün küme kimliğini tutar. `id[p] == id[q]` ise çift zaten bağlıdır; değilse kimliği `id[p]` olan bütün elemanlar `id[q]` yapılır. Bölüm 2.5'teki tabloda 4-3, 3-8, 6-5, 9-4, 2-1, 8-9, 5-0, 7-2, 6-1 dizisi için adımlar var. Son durumda "1 ile 8 bağlı mı?" sorusunun cevabı hayırdır, çünkü `id[1] = 1` ve `id[8] = 8`.

Soru 3 (derste sözlü, 00:32:33): 600 000 kelimenin her biri 1 saniyede karşılaştırılırsa sıralı arama yaklaşık 166 saat sürer. Bu kaç gün eder?

> **Çözüm:** Derste "neredeyse bir hafta" denildi. Hesap: $600\,000 / 86\,400 \approx 6{,}94$ gün.

Soru 4 (derste sözlü, 00:38:37): 6 ve 18'in asal çarpanları nelerdir? 1 asal sayı mıdır?

> **Çözüm:** 6'nın asal çarpanları 2 ve 3; 18'in asal çarpanları 2 ve 3'tür ($18 = 2 \cdot 3^2$). 1 asal değildir, çünkü asal sayı tanımı 2 ya da daha büyük olmayı gerektirir. Ortak asal çarpanların çarpımı $2 \cdot 3 = 6$ OBEB'dir.

Soru 5 (derste sözlü, 00:39:56): "Asal çarpanlarına ayır, ortak asal çarpanları çarp" yöntemi bir algoritma mıdır?

> **Çözüm:** Bu hâliyle değildir. Girdisi ve çıktısı vardır, ama "asal çarpanlarını bul" adımının nasıl yapılacağı tanımlanmamıştır. Algoritmanın her adımı açık ve kesin olmalıdır.

Soru 6 (derste sözlü, 00:48:00): Öklid algoritmasının doğru çalıştığını göstermek için n'nin mutlaka bir noktada 0 olacağını nasıl ispatlarsınız?

Derste bir öğrenci, OBEB varsa bölmelerin bir noktada kalansız biteceğini, aralarında asal sayılarda da en küçük sayıya doğru gidileceğini söyledi. Hoca soruyu düşünülmek üzere bıraktı.

> **Çözüm (derste verilmedi):** Her adımda yeni n değeri $r = m \bmod n$'dir ve kalanın tanımı gereği $0 \leq r < n$'dir. Yani n'nin değeri her adımda kesin olarak azalır ve hiçbir zaman negatif olmaz. Negatif olmayan tam sayılardan oluşan, kesin azalan bir dizi sonsuza kadar sürmez; en fazla başlangıçtaki n kadar adımda 0'a ulaşır. Sonucun doğruluğu için de $\gcd(m, n) = \gcd(n, m \bmod n)$ eşitliği gerekir: $m = qn + r$ olduğundan, m ile n'yi bölen her sayı $r = m - qn$'yi de böler; n ile r'yi bölen her sayı da $m = qn + r$'yi böler. İki çiftin ortak bölenleri aynıdır, dolayısıyla en büyükleri de aynıdır. $n = 0$ olduğunda $\gcd(m, 0) = m$'dir.

Soru 7 (derste sözlü, 01:37:49): $n!$'den daha kötü bir büyüme oranı ne olabilir?

> **Çözüm:** Bir öğrenci "$n!$'in faktöriyeli" dedi. Hocanın verdiği cevap n^n'dir. $n! = n \cdot (n-1) \cdots 1$ çarpımındaki her çarpan n'den küçük ya da eşit olduğu için $n! \leq n^n$'dir.

Soru 8 (derste sözlü, 02:04:48): $n = 10\,000$ için n^2 mertebesindeki bir algoritma kaç işlem yapar?

> **Çözüm:** $10\,000 \cdot 10\,000 = 10^8$, yani 100 milyon işlem.

Dersin Toparlaması

Bu derste algoritma tasarımının ve analizinin temelleri iki yönden ele alındı. Tasarım tarafında bağlantı problemi, doğru problem tanımı ve doğru veri yapısı seçiminin çözümü ne kadar basitleştirdiğini gösterdi: "Hangi yolla bağlı?" yerine "bağlı mı?" sorusu sorulunca, tek bir dizide küme kimlikleri tutarak problem çözüldü. Kelime arama örneği, girdinin sıralı olduğunu kullanan ikili aramanın 166 saati 20 saniyeye indirdiğini gösterdi. Öklid algoritması ise her adımı açık tanımlı bir algoritmanın, adımları muğlak ortaokul yönteminden farkını ortaya koydu.

Analiz tarafında deneysel ölçümün gerçekleştirmeyi ve ortamı ölçtüğü, bu yüzden teorik analizin tercih edildiği anlatıldı. İşlem maliyetleri sayılarak çalışma süresi n'ye bağlı bir fonksiyon olarak yazıldı. Büyüme oranları sıralandı, Big O (üst sınır), Big Omega (alt sınır) ve Big Theta (iki taraflı sınır) biçimsel olarak tanımlandı. Bu ders boyunca algoritmalar en kötü durum ve Big O üzerinden incelenecek. Sıralı arama $O(n)$, ikili arama $O(\log n)$; bubble, selection ve insertion sort en kötü durumda $O(n^2)$'dir. Hoca gelecek hafta Fibonacci örneği üzerinden üstel karmaşıklığa ve dinamik programlamaya değineceğini söyledi. Öğrencilerden dersi tekrar izlemelerini, daha önce yazdıkları algoritmaların en kötü, en iyi ve ortalama durumlarını kendilerinin analiz etmelerini ve internetteki algoritma problemlerini çözmeye çalışmalarını istedi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.